

NYCU-EE IC LAB – Fall 2025

Lab01 Exercise

Design: Multi-Packet Channel Arbiter(MPCA)

Data Preparation

1. Extract files from TA's directory:

```
% tar -xvf ~iclabTA01/Lab01_2025F.tar
```

Design Description and Examples

In today's high-speed era, how to implement an efficient multi-channel communication system has become an important and widely discussed topic. When multiple packets simultaneously request access to different channels, the system needs an intelligent arbitration mechanism to effectively allocate limited resources, maintain load balance, and improve overall performance.

In this lab, we will design a simplified **Multi-Packet Channel Arbiter (MPCA)** to simulate such a scenario. Your design should be able to prioritize and sort multiple packets and, while considering the current channel loads, make reasonable channel allocation decisions to achieve efficient and specification-compliant resource scheduling.

✓ Description of Lab

➤ Input:

You will receive four sets of inputs:

- **packets** (128 bits) represents 8 **encrypted** packets, which means { **packet[7]**, **packet[6]**, **packet[5]**, **packet[4]**, **packet[3]**, **packet[2]**, **packet[1]**, **packet[0]** }. After **decrypted**, each packet encodes various attributes such as request validity, quality of service (QoS), length, congestion level, preferred channel, source hint, and a mode indicator. Each packet (16 bits) encodes with the following structure:
 - [15]: req_valid — whether this packet is requesting service
 - [14:13]: qos — quality of service
 - [12:9]: pkt_len — length of packet
 - [8:7]: congestion — congestion indicator
 - [6:5]: prefer_ch — preferred target channel (0~2)
 - [4:2]: src_hint — additional hint about source
 - [1]: mode — indicates how to interpret fields (0: unsigned, 1: signed)

- [0]: reserved — reserved bit for alignment, ignored
- **channel_load** (12 bits) is a 12-bit input representing the **initial** load condition of the three channels at the start of each case. It is organized as three 4-bit fields corresponding to **channel_load[2]**, **channel_load[1]**, and **channel_load[0]**, with each field specifying **initial load value in the range 0 to 15**, where each field specifies the current number of packets already present in the respective channel before any new allocation takes place.
- **channel_capacity** (9 bits) is a 9-bit input that defines the maximum allowable number of packets for three channels in the current case. It is structured as three 3-bit fields, corresponding to **channel_capacity[2]**, **channel_capacity [1]**, and **channel_capacity [0]**, with each field specifying a **capacity value in the range 0 to 7**.
- **KEY** is a 64 bit master key in SPECK, which is first split into four 16-bit words. These words are then fed through an ARX-based key schedule—using bit-rotations, modular addition, and XOR with a round constant—to produce four 16-bit subkeys.

➤ **Packet Encryption & Decryption**

In this lab, we use SPECK32/64 to do the encryption/decryption. SPECK32/64 is a lightweight symmetric block cipher published by the U.S. National Security Agency (NSA) for resource-constrained embedded systems and IoT devices. It uses only three basic operations—Addition, Rotation, and XOR (ARX)—to minimize hardware complexity and maximize software speed.

In SPECK32/64, each plaintext block is 32 bits (two 16-bit words) and the key is 64 bits. Encryption runs through multiple rounds (e.g. 22 rounds) of ARX mixing. In each round, a round key—derived via a simple key schedule—is applied by: 1. Rotating and adding the two words, 2. XOR with the round key, 3. Performing another rotation and XOR.

We begin with eight 16-bit packet words—totaling 128 bits—and protect them using the SPECK32/64 block cipher. We use a **variable 32-bit master KEY** and perform **4 rounds** of ARX operations (where each round consists of Addition, bitwise Rotation, and XOR). After encrypting all packet data, we immediately decrypt it to verify that the original words are fully recovered.

First, we take our sequence of eight 16-bit words ({ **packet[7]**, **packet[6]**, **packet[5]**, **packet[4]**, **packet[3]**, **packet[2]**, **packet[1]**, **packet[0]** }) and group them into four 32-

bit units. For each block index b (from 0 to 3). Assign $x0 = \text{packet}[2b]$ as the low half, and $y0 = \text{packet}[2b+1]$ as the high half and pack these into a single 32-bit word

$$\text{blk_plain}[b] = \{ \text{packet}[2b+1], \text{packet}[2b] \}$$

After this step, our entire 128-bit packet is represented as four 32-bit plaintext blocks $\text{blk_plain}[0]$ through $\text{blk_plain}[3]$, ready for encryption.

The next step is to generate the key to do encrypt and decrypt, in SPECK's key schedule, we don't just use a single 16-bit subkey k — we also maintain three additional 16-bit “state” words in an array $l[0\dots2]$. Here's how they're initialized and used:

- **Initialization to generate key**

- Take the 48-bit master key and split off its lowest 16 bits as $k (K_0)$.
- The remaining upper 32 bits are divided into three 16-bit words:
 - $l[2] \leftarrow$ the highest 16 bits of the key
 - $l[1] \leftarrow$ the next 16 bits
 - $l[0] \leftarrow$ the lowest 16 bits of the remaining key

Thus the full 64-bit key is represented as one “current key” k plus three “key-state” words $l[0]$, $l[1]$, and $l[2]$.

- **Subkey Scheduling**

For each round index r :

1. Emit the current value of k as the round key K_r .
2. Compute a temporary value

$$t = (\text{ROR}_7(k) + l[r \bmod 3]) \bmod 2^{16} \oplus r$$

mixing k with one of the l -words via ARX operations, ROR_7 rotates right by 7 bits.

3. Update the same $l[r \bmod 3]$ with

$$l[r \bmod 3] := \text{ROL}_2(l[r \bmod 3]) \oplus t$$

In this exercise, since we only perform it four times, this step is optional.

4. Set **k=t** so the next round uses the new subkey of K_{r+1} .

- **Encryption**

After you get all round of K_r and blk_plain, then we can do the next step to encrypt, in this exercise let

$$\alpha=7, \beta=2, R=4, \text{round_key} = \{K0, K1, K2, K3\}$$

For ARX transformation, each round $r=0 \dots 3$ consists of two step:

1. **Mix & Key-Whiten**

$$x_{r+1} = (\text{ROR}_7(x_r) + y_r) \bmod 2^{16} \oplus K_r.$$

2. **Diffusion**

$$y_{r+1} = \text{ROL}_2(y_r) \oplus x_{r+1}$$

After four rounds, we can get the encrypted packet $\{y_4, x_4\}$, and do it four times and concatenating the results, we can get the 128-bit cipher(encrypted) packet, and then we can successfully create the pattern and send it into our design.

In our design, we will get the encrypted packet and need to recover it, so we have to do the **Inverse ARX Operations**, which means we decrypt each cipher block by reversing the four rounds (from $r=3$ down to 0). For each round:

1. **Recover the high half**

$$y_r = \text{ROR}_2(y_{r+1} \oplus x_{r+1})$$

2. **Recover the low half**

$$x_r = \text{ROL}_7((x_{r+1} \oplus K_r) - y_r) \bmod 2^{16}$$

After four rounds, we can get the decrypted packet $\{y_0, x_0\}$, and do it four times and concatenating the results, we can get the 128-bit plain(decrypted) packet, and then we can successfully get the original information of packets.

➤ **Priority Score Calculation:**

After the packets arrive, the system will examine the information contained in each packet to determine its attributes and current status, and then compute a priority score used for ordering the packets. This priority score takes into account multiple fields within the packet, including QoS, packet length, congestion status, and a source hint. These field values collectively influence the packet's priority and subsequent channel allocation decisions.

The formula of the priority score is as follows:

$$\text{priority_score} = (\text{qos} - 2) * 4 + (\text{pkt_len} - 8) * (-2) + (1 - \text{congestion}) * 3 + (\text{src_hint} - 4)$$

Through this calculation, packets with higher QoS, shorter lengths, lower congestion levels, or a source hint closer to a specific target will generally receive a higher priority.

It is important to note that how these field values are interpreted depends on the mode field of the packet:

- When **mode equals 0**, all field values are interpreted and computed as **unsigned integers**.
- When **mode equals 1**, the field values are interpreted as 2's complement **signed integers**, meaning that the most significant bit of the field serves as the sign bit.

➤ Packet Arbitration and Allocation

After the packets enter the MPCA, the system must first **sort all packets to determine their processing priority order**.

The sorting follows these rules:

- First, compare the **req_valid** field: packets with req_valid = 1 are considered valid and don't need to care when req_valid = 0.
- If two packets have the same req_valid value, compare their **priority scores**: the packet with the higher score has higher priority.
- If the priority scores are equal, the packet which has **smaller index** has higher priority.

After sorting, the MPCA allocates channels to packets in order. For each packet, the system applies the following allocation policy:

- If req_valid = 0, the packet is immediately marked as unallocated (2'b11).
- Otherwise, the arbiter first attempts to allocate the packet to its preferred channel if that channel has not reached its capacity.
- If the preferred channel is full, fallback allocation is performed using a **dynamic round-robin algorithm**:
 - When fallback occurs for the first time in the current test case, the **preferred channel of that packet is recorded as the initial pivot** for all subsequent fallback searches in this case.
 - Fallback searches proceed in **cyclic order** starting from this pivot:

$$\{ (\text{pivot}+0) \% 3, (\text{pivot}+1) \% 3, (\text{pivot}+2) \% 3 \}$$

- If fallback succeeds, the packet is allocated to the first available channel found; if all channels are full, the packet is marked as unallocated (2'b11).

After each fallback attempt, the pivot for the next fallback search is updated as part of the dynamic round-robin process:

- If the fallback succeeded:

$$\text{next_pivot} = (\text{pivot} + 1) \% 3$$

- If the fallback failed:

$$\text{next_pivot} = (\text{pivot} + 2) \% 3$$

For example, suppose the first fallback occurs when the preferred channel is 2, so the dynamic round-robin search order is $2 \rightarrow 0 \rightarrow 1$.

If this fallback succeeds, then the next fallback search will start from channel 0 starting from channel $(2 + 1) \% 3 = 0$, so the next search order would be $0 \rightarrow 1 \rightarrow 2$.

Instead, if this fallback fails, which means no channels are available, then the next fallback search will start from two channels ahead: starting from channel $(2 + 2) \% 3 = 1$, so the next search order would be $1 \rightarrow 2 \rightarrow 0$.

➤ Mask Policy

After the initial channel allocation is completed, the MPCA performs an additional check on each packet that has successfully been assigned a channel to determine whether it meets the mask condition. This check calculates a mask score based on the packet's attributes, preferred channel, source hint, and the current channel load, and then compares it to a threshold value.

In this lab, the conceptual formula for mask_score is as follows:

$$\text{Mask_score} = ((\text{priority_score} \& 0x6) + \text{prefer_ch} + (\text{src_hint} \wedge 0x3) + \text{channel_load}[\text{ch}]) \% 10$$

The mask score extracts bits from the packet's priority score, adds the preferred channel, a modified source hint, and the initial channel load, here ch means prefer_ch, then applies modulo 10 to produce a normalized value for comparison.

$$\text{Threshold} = 7 + (\text{channel_load}[\text{ch}] / 3)$$

If a packet's mask_score is greater than or equal to the threshold, the packet is marked as mask failed. It is important to note that **even if a packet is marked as**

mask failed, it still occupies its assigned channel resource and the allocation is not released or reassigned.

➤ Global Rebalance & Output

After completing the initial channel allocation and mask policy, the MPCA must check whether the channel loads are balanced.

In this step, the system calculates the total load for each channel, **including both the initial channel_load and the number of newly allocated packets** and identifies the channel with the highest load. It then compares this channel's total load with the combined load of the other two channels.

If the **total load of one channel exceeds half the sum of the other two channels' loads**, a rebalance operation is triggered. The system will attempt to move one packet from the overloaded channel to another channel to improve load balance. The rebalance operation follows these constraints:

- Only the packet with the lowest priority in the overloaded channel is considered.
- Packets marked as mask failed are not eligible for rebalance.
- Rebalance searches proceed in **cyclic order** starting from this current channel

$$\{ (\text{channel} + 0) \% 3, (\text{channel} + 1) \% 3, (\text{channel} + 2) \% 3 \}$$

- A rebalance operation is only considered successful if, after moving a packet, the target channel still has available capacity, **and** its total load remains strictly less than 16 in the same time.

If rebalance is successfully performed, the packet's channel assignment is updated; otherwise, the selected packet will be drop. To simplify this exercise, **the global rebalance will be executed only once whether its result succeeds or fails.**

grant_channel is the final output signal required by the assignment, representing the channel allocation decisions for eight packets. This signal is composed of the channel indices for all eight packets, where each index indicates the channel to which a packet is ultimately assigned (or the special value 3 if the packet is not allocated). The channel indices are concatenated in order to form a single output vector.

Output(16-bit)							
MSB				LSB			
Packet7	Packet6	Packet5	Packet4	Packet3	Packet2	Packet1	Packet0
2-bit	2-bit	2-bit	2-bit	2-bit	2-bit	2-bit	2-bit

Ideal Example:

packets = **0xe040_5cd2_4527_5495_524a_3c2a_791a_9a33**

KEY = **0xe750_3a4f_51c0_bb1a**

channel_load = {*channel* 0: **2**, *channel* 1: **7**, *channel* 2: **D**}

channel_capacity = {*channel* 0: **4**, *channel* 1: **4**, *channel* 2: **6**}

1. Decrypt

The first step we need to find the subkey, and we know $K_0 = bb1a$, by the key scheduling formula

$$K_1 = (ROR_7(bb1a) + l[0]) \bmod 2^{16} \wedge 0 = 8736$$

$$K_2 = (ROR_7(8736) + l[1]) \bmod 2^{16} \wedge 1 = a75c$$

$$K_3 = (ROR_7(a75c) + l[2]) \bmod 2^{16} \wedge 2 = a09c$$

Then we can do the decryption, here we take the lower 32 bit for example ({packet1, packet0}):

$$y_3 = ROR_2(791a \wedge 9a33) = 78ca$$

$$x_3 = ROL_7((9a33 \wedge a09c) - 78ca) \bmod 2^{16} = f2e0$$

$$y_2 = ROR_2(78ca \wedge f2e0) = a28a$$

$$x_2 = ROL_7((f2e0 \wedge a75c) - a28a) \bmod 2^{16} = 9959$$

$$y_1 = ROR_2(a28a \wedge 9959) = cef4$$

$$x_1 = ROL_7((9959 \wedge 8736) - cef4) \bmod 2^{16} = bda7$$

$$y_0 = ROR_2(cef4 \wedge bda7) = \textcolor{red}{dcd4}$$

$$x_0 = ROL_7((bda7 \wedge bb1a) - dcd4) \bmod 2^{16} = \textcolor{red}{f494}$$

So we can find that the original {packet1, packet0} = dcd4_f494, and do the same thing on other packet, we can recover the original packet

original_packets = **0xb95e_9b24_af5e_90d2_a020_faa6_dcd4_f494**

2. Decode

	req_valid	qos	pkt_len	congestion	prefer_ch	src_hint	mode
Packet0	1	3	10	1	0	5	0
Packet1	1	2	14	1	2	5	0
Packet2	1	3	13	1	1	1	1
Packet3	1	1	0	0	1	0	0
Packet4	1	0	8	1	2	4	1
Packet5	1	1	7	2	2	7	1
Packet6	1	0	13	2	1	1	0
Packet7	1	1	12	2	2	7	1

3. Calculate Priority Scores

I. Take packet 0 for example, its mode is 0, so we don't need to do sign calculation

mode 0

qos = 3 → 3

pkt_len = 10 → 10

congestion = 1 → 1

src_hint = 5 → 5

So the priority score is $(3-2)*4+(10-8)*(-2)+(1-1)*3+(5-4) = 1$

II. Take packet 7 for example, its mode is 1, so we need to do sign calculation, take the highest bit as signed bit

mode 1

qos = 1 → 1 (2'b01)

pkt_len = 12 → -4 (4'b1100)

congestion = 2 → -2 (2'b10)

src_hint = 7 → -1 (3'b111)

So the priority score is $(1-2)*4+(-4-8)*(-2)+(1-(-2))*3+(-1-4) = 24$

After calculation and **sorted as the rule**, you can get as the following list:

rank	packet	Priority Scores	req_valid	Prefer_ch
Rank0	Packet7	24	1	2
Rank1	Packet4	16	1	2
Rank2	Packet3	11	1	1
Rank3	Packet2	7	1	1
Rank4	Packet5	2	1	2
Rank5	Packet0	1	1	0
Rank6	Packet1	-11	1	2
Rank7	Packet6	-24	1	1

4. Packet Arbitration and Allocation

sorted order	Req_valid	Priority Score	Allocation channel
Packet0	1	1	0
Packet1	1	-11	2
Packet2	1	7	1
Packet3	1	11	1
Packet4	1	16	2
Packet5	1	2	2
Packet6	1	-24	1
Packet7	1	24	2

In this case, we don't have fail allocation, so we don't need to care about fallback policy, just assign the channel in order

5. Mask

I. Take packet 0 for example, its priority score is 1, **prefer_ch** is 0, src_hint is 5, **allocation_channel** is 0, which **initial load** is 2:

$$\text{mask_score} = \{ (1 \& 0x6) + 0 + (0 \wedge 0x3) + 4 \} \% 10 = 8$$

$$\text{threshold} = 7 + (2 / 3) = 7$$

in such case, the mask_score is larger(equal) than threshold, so its mask **failed**

II. Take packet 6 for example, its priority score is -24, **prefer_ch** is 1, src_hint is 1, **allocation_channel** is 1, which **initial load** is 13:

$$\text{mask_score} = \{ (-24 \& 0x6) + 1 + (1 \wedge 0x3) + 13 \} \% 10 = 0$$

$$\text{threshold} = 7 + (13 / 3) = 11$$

in such case, the mask_score is smaller than threshold, so its mask **success**

6.Global Rebalance

I. calculate total channel load, after first allocation

$$\text{Channel_load_0} = 2 + 1 = 3$$

$$\text{Channel_load_1} = 7 + 3 = 10$$

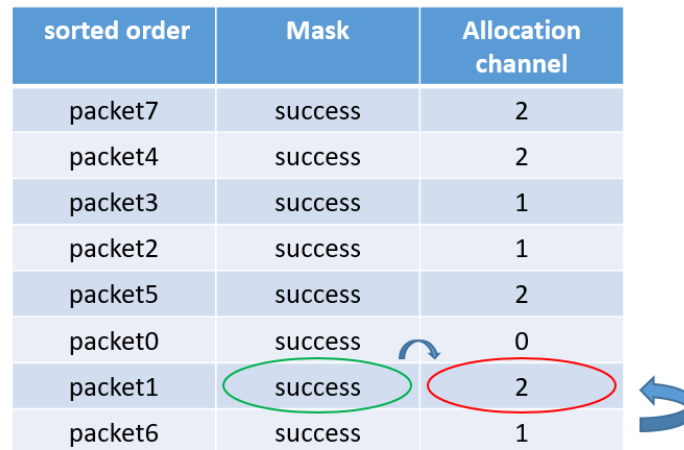
$$\text{Channel_load_2} = 13 + 4 = 17$$

Apparently, the maximum load **Channel_load_2** is larger than half sum of other two, so we need to do rebalance

II. find packet to be reassigned to different channel

By the sorted list, we can know that our finding order is $6 > 1 > 0 > 5 > 2 > 3 > 4 > 7$

sorted order	Mask	Allocation channel
packet7	success	2
packet4	success	2
packet3	success	1
packet2	success	1
packet5	success	2
packet0	success	0
packet1	success	2
packet6	success	1



III. find alternative channel

In this case, the original allocation channel is 2, so according to the alternative principle

$$\text{Alternative_channel_1} = (2 + 1) \% 3 = 0$$

$$\text{Alternative_channel_2} = (2 + 2) \% 3 = 1$$

Also, we know that the capacity of other two is not full, and after allocation they still have received fewer than 16 packets, so we can successfully put the packet1 into the channel 0, which is the first alternative channel, and the final answer will become as the following:

sorted order	grant channel
Packet0	0
Packet1	0
Packet2	1
Packet3	1
Packet4	2
Packet5	2
Packet6	1
packet7	2

The out will become 16'b1001101001010000 ([15:14] is packet7, [1:0] is packet0)

Ugly Example:

packets = 0x5649_5a15_8f3b_eee6_06fb_164a_56ac_c1e5

KEY = 0xaeed_bd3c_30ab_f5ae

channel_load = {channel 0: **D**, channel 1: **1**, channel 2: **8**}

channel_capacity = {channel 0: **4**, channel 1: **0**, channel 2: **3**}

1. After Decrypting, Decoding, calculating and sorting...

rank	packet	Priority Scores	req_valid	Prefer_ch
Rank0	Packet7	3	1	1
Rank1	Packet5	-6	1	1
Rank2	Packet0	-9	1	1
Rank3	Packet1	7	0	1
Rank4	Packet2	12	0	2
Rank5	Packet3	1	0	1
Rank6	Packet4	-6	0	1
Rank7	Packet6	9	0	2

2. Channel allocation

In this case, we meet the worst situation that all the required packets want to be allocated into a full capacity channel, so we need to do the dynamic round-robin process, shown as the following picture, the pivot start from the packet7 which prefer_ch is 1:

	7	5	0
Pivot 0	1	2	0
Pivot 1	2	0	1
Pivot 2	0	1	2

3. Mask and Global rebalance

By the mask calculation, we know that all the required packets meet the mask policy, in the same time, we found that the loading of channel 0 is heavier than the half sum of other two channels, so we need to do the global rebalance.

sorted order	Mask	Allocation channel
Packet7	success	2
Packet5	success	2
Packet0	success	0
Packet1	x	3
Packet2	x	3
Packet3	x	3
Packet4	x	3
packet6	x	3

In this case, the original allocation channel is 0, so according to the alternative principle

$$\text{Alternative_channel_1} = (0 + 1) \% 3 = 1$$

$$\text{Alternative_channel_2} = (0 + 2) \% 3 = 2$$

Also we know that the capacity of channel 1 is full and the channel 2 is not full yet, so we can successfully put the packet0 into the channel 2, which is the second alternative

channel, and the final answer will become as the following:

sorted order	grant channel
Packet0	2
Packet1	3
Packet2	3
Packet3	3
Packet4	3
Packet5	2
Packet6	3
packet7	2

The out will become **16'b1011101111111110** ([15:14] is packet7, [1:0] is packet0)

The summary of the description and specifications are as followings:

Input Signal	Bit Width	Description
packets	128	Encodes eight packets, each 16 bits, containing fields such as request validity, QoS, packet length, congestion, preferred channel, source hint, and mode.
channel_load	12	Initial load for three channels before allocation, with 4 bits per channel.
channel_capacity	9	Maximum allowed allocation for three channels, with 3 bits per channel.
KEY	64	The master key to decrypt the packets

Output Signal	Bit Width	Description
grant_channel	16	The final output signal representing the channel assignments for eight packets. 2'b00 : allocate to channel 0 2'b01 : allocate to channel 1 2'b10 : allocate to channel 2 2'b11 : fail assign, drop the packet

Inputs

1. The input signals **packets** is 128-bit, you need to do the SPECK32/64 to decrypt.
2. The input signal **channel_load** is a 12-bit representing the **initial** load condition of the three channels at the start of each case, remind that **after allocation, each channel_load might not be only 4 bits**.
3. The input signal **channel_capacity** is a 9-bit representing the maximum allowed allocation for three channel.
4. The input signal **KEY** is a 64-bit which use as the master key for SPECK algorithm to decrypt the packets.

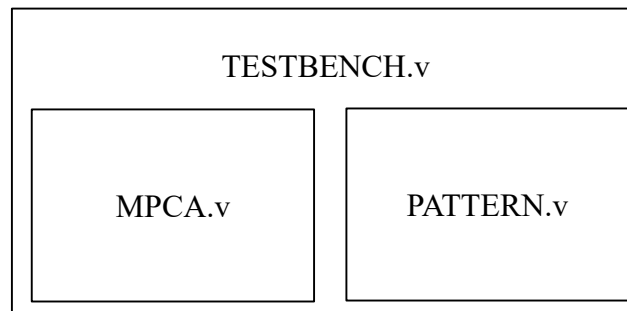
Outputs

The output signal **grant_channel** is a 16-bit number which is concatenation of the allocated channel from packet7 to packet0. This represents the correct result.

Specifications

1. Top module name : MPCA(File name: MPCA.v)
2. After synthesis, check the “MPCA.area” and “MPCA.timing” in the folder “Report”. **The area report is valid only when the slack in the end of “MPCA.timing” is “MET”.**
3. The synthesis result **cannot** contain any **latch**.
Note: You can check if there is a latch by searching the keyword “**Latch**” in 02_SYN/syn.log
4. **Hardcoding answers** or using a lookup table based on the test case index is prohibited; Your design must compute outputs correctly based on the actual inputs. You will get **0 score** if you use this method, **no second chance**.
5. **Synopsys’ IP and any comment of // synopsys * is prohibited** in this lab.
6. Only English comments are allowed
7. In this lab, **you can adjust your clock cycle time but it should be less than 36.5 ns, and you can only adjust it in unit of 0.1ns. Consequently, make sure to key in your clock cycle time when you submit.** It’s means that the TA will demo your design under this clock cycle time.
8. **There will be some hidden case in this Lab, that means you need to consider some conditions that may occur beyond the provided open cases!**

Block Diagram



Grading Policy

The performance is determined by the area of your design. The less area your design has, the higher grade you get. Try to reach better performance by thinking your architecture before coding. **Just a reminder: only those who pass the first demo will be considered for the best code in this Lab.**

Function Validity: 70%

Performance: $\text{cycle_time}^2 * \text{area}$ 30%

If you fail Lab01 at first demo, and pass at **second demo, you will only get 50 points.**

Get no score if you fail both first and second demo. Note that you will get **0 score** if you are **found plagiarism** at your code.

Note

1. Tar all your design by run the command **Lab01/09_SUBMIT/00_tar**
2. Submit your design through **Lab01/09_SUBMIT/01_submit**
 - a. 1st_demo deadline: **2025/09/15(Mon.) 12:00:00**
 - b. 2nd_demo deadline: **2025/09/17(Wed.) 12:00:00**
3. If your file **violates the naming rule**, you will **lose 5 points**.
4. Don't use any wire/reg/submodule/parameter name called ***error***, ***Congratulations***, ***latch*** or ***FAIL*** otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g.: error_note is forbidden.
5. Please begin your assignment as early as possible to avoid unpredictable server problems. Be sure to use `./02_check` to download and verify your submission files. Also, do **not** use System Verilog syntax—if your demo fails due to any of the issues above, the TAs will not be held responsible.

Be careful about all details!

Template folders and reference commands:

In demo, the reference commands is:

1. 01_RTL (RTL simulation):

`./01_run_vcs_rtl`

2. 02_SYN/ (Synthesis):

`./01_run_dc_shell`

(Check **latch** by searching the keyword “**Latch**” in 02_SYN/syn.log)

(Check the design’s timing in /Report/ MPCA.timing)

(Check the design’s area in /Report/ MPCA.area)

3. 03_GATE/ (Gate-level simulation):

`./01_run_vcs_gate`

4. 09_SUBMIT/ (submit your files):

`./00_tar your_cycle_time`

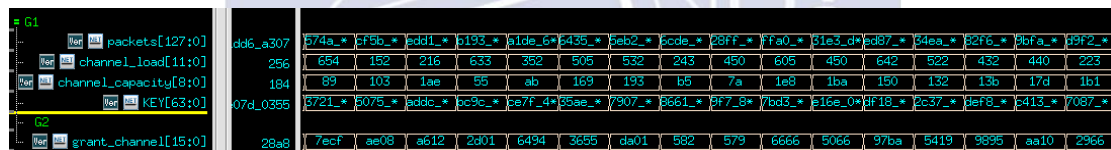
`./01_submit 1st_demo/2nd_demo`

`./02_check 1st_demo/2nd_demo`

You can key in `./09_clean_up` to clear all log files and dump files in each folder

Example Waveform

Input and output signal:



Hint

Hint1: Try to use **behavior modeling description** instead of gate level description.

Hint2: Try to use **submodule** rather than copy and paste to simply your design. (not necessary in this lab)

```
// -----  
// Example for using submodule  
// BBQ bbq0(.meat(meat_0), .vegetable(vegetable_0), .water(water_0),.cost(cost[0]));  
// -----
```

Hint3: Pattern provided by TA will cover **only some simple cases**, you can try to write **your own input / output file** by yourself. Here is the format how TA will read in PATTERN, take care to use **hexadecimal** numbers to generate the pattern, or your terminal will display the following warning and fail:

```

input.txt format (using hex)
1. [PATTERN_NUM]
repeat(PATTERN_NUM)
  1. [packet0]
  2. [packet1]
  3. [packet2]
  4. [packet3]
  5. [packet4]
  6. [packet5]
  7. [packet6]
  8. [packet7]
  9. [KEY]
 10. [channel_load0]
 11. [channel_load1]
 12. [channel_load2]
 13. [channel_capacity0]
 14. [channel_capacity1]
 15. [channel_capacity2]

```

```

output.txt format
repeat(PATTERN_NUM)
  1. [grant_channel0]
  2. [grant_channel1]
  3. [grant_channel2]
  4. [grant_channel3]
  5. [grant_channel4]
  6. [grant_channel5]
  7. [grant_channel6]
  8. [grant_channel7]

```

```

Warning: Too many digits in hexadecimal number '13', truncating to target size 4 bits
at time 44000 in file PATTERN.v line 145

Warning: Too many digits in hexadecimal number '15', truncating to target size 4 bits
at time 44000 in file PATTERN.v line 145

Warning: Too many digits in hexadecimal number '11', truncating to target size 4 bits
at time 44000 in file PATTERN.v line 145

```

You can check input.txt and PATTERN.v in 00_TESTBED as a reference, and choose to write either c++/python or Verilog code for generating other cases.

Hint4: If you want to change your cycle time, make sure you change the following parts in your directory, and run the 03_GATE to check

In 00_TESTBED/PATTERN.v, change CYCLE_TIME

```

18 //#####
19
20 `define CYCLE_TIME 36.5
21

```

In 02_SYN/syn.tcl, change CYCLE

```

9 #=====
10 set DESIGN "MPCA"
11 set CYCLE 36.5
12 set INPUT_DLY [expr 0.5*$CYCLE]
13 set OUTPUT_DLY [expr 0.5*$CYCLE]
14

```

Hint5: TA is providing the implementation of the pseudo code of algorithm for this lab as a reference for testing. Of course, if you come up with a better algorithm and your solution also meets the requirements of the problem, you are welcome to implement it yourself.

Algorithm 1: MPCA Arbitration and Allocation Algorithm

Input: 8 packets; 3 channels (load, capacity)

Output: grant_channel assignments

```

1 foreach test case do
2   Initialize random packets and channels;
3   Encrypt the packet and send it to the MPCA.v;
4   In MPCA.v, decrypt the encrypted packet by Algorithm 2;
5   foreach decrypted packet do
6     Interpret fields (signed/unsigned by mode);
7     Compute priority_score;
8   Sort packets by req_valid, priority_score (desc), index (asc);
9   foreach decrypted packet in sorted order do
10    if preferred channel has capacity then
11      Allocate to preferred channel;
12    else
13      Attempt round-robin fallback from pivot;
14      After fallback then Update pivot:
        • Success: pivot = (pivot + 1) mod 3
        • Fail: pivot = (pivot + 2) mod 3
15  foreach allocated packet do
16    Compute mask_score = ((priority_score & 0x6) + preferred_ch +
      (src_hint ⊕ 0x3) + channel.load) mod 10;
17    Compute threshold =  $7 + \frac{\text{initial\_channel\_load}}{3}$ ;
18    if mask_score ≥ threshold then
19      Mark as mask_failed;
20  Compute total load for each channel;
21  if any channel overloaded then
22    Find lowest-priority, mask-success packet in overloaded channel;
23    Attempt to reassign to other channel;
24    if cannot move then
25      Drop packet (grant_channel = 3);
26  Output grant_channel results;

```

Algorithm 2: SPECK32/64 Encryption & Decryption (4 rounds)

Input: `pkt_words[0..7]`: eight 16-bit packet words,
KEY: 64-bit master key,
ROUNDS = 4
Output: `cipher_pkt[127 : 0]`: 128-bit ciphertext,
`plain_pkt[127 : 0]`: recovered plaintext

```
1 for  $b \leftarrow 0$  to 3 do
2    $x \leftarrow \text{pkt\_words}[2b]; \quad y \leftarrow \text{pkt\_words}[2b + 1];$ 
3    $\text{blk\_plain}[b] \leftarrow \{y, x\}$ 
4 Slice KEY into
5  $k = \text{KEY}[15:0],$ 
6  $l[0] = \text{KEY}[31:16],$ 
7  $l[1] = \text{KEY}[47:32],$ 
8  $l[2] = \text{KEY}[63:48].;$ 
9 for  $r \leftarrow 0$  to 3 do
10   $\text{rk}[r] \leftarrow k;$ 
11   $t \leftarrow (\text{ROR}_7(k) + l[r \bmod 3]) \oplus r;$ 
12   $l[r \bmod 3] \leftarrow \text{ROL}_2(l[r \bmod 3]) \oplus t;$ 
13   $k \leftarrow t$ 
14 for  $b \leftarrow 0$  to 3 do
15   $(x, y) \leftarrow \text{blk\_plain}[b];$ 
16  for  $r \leftarrow 0$  to 3 do
17     $x \leftarrow (\text{ROR}_7(x) + y) \oplus \text{rk}[r];$ 
18     $y \leftarrow \text{ROL}_2(y) \oplus x$ 
19   $\text{blk\_cipher}[b] \leftarrow \{y, x\}$ 
20  $\text{cipher\_pkt} \leftarrow \{\text{blk\_cipher}[3], \dots, \text{blk\_cipher}[0]\}$ 
21 for  $b \leftarrow 0$  to 3 do
22   $(x', y') \leftarrow \text{blk\_cipher}[b];$ 
23  for  $r \leftarrow 3$  down to 0 do
24     $\text{tmpY} \leftarrow y' \oplus x';$ 
25     $y' \leftarrow \text{ROR}_2(\text{tmpY});$ 
26     $\text{tmpX} \leftarrow x' \oplus \text{rk}[r];$ 
27     $x' \leftarrow \text{ROL}_7(\text{tmpX} - y')$ 
28   $\text{blk\_dec}[b] \leftarrow \{y', x'\}$ 
29  $\text{plain\_pkt} \leftarrow \{\text{blk\_dec}[3], \dots, \text{blk\_dec}[0]\}$ 
```

